How Javascript work Internally

JS Engine : program that execute Javascript code

JS Engine

: Heap store All Reference data, Objet, array, etc

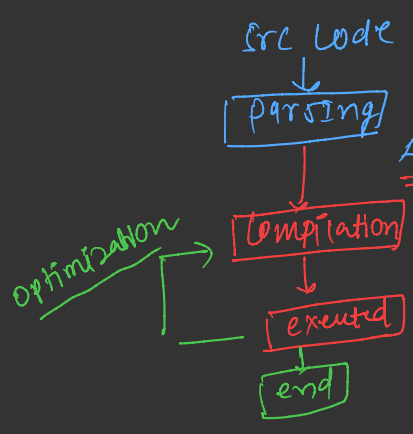
gec → global execution context

* Callstack
  + global Execution Context creation only 1 gec created a JS file.
  - variable declation + Initiation

* Compilation v/s Interpretion
  + Compile src → portable file → executed on machine
  + Interpreted → line by line code executed. (slow process)

* modern JS use (JIT)
  { JIT = JUST In time compilation.

* How JIT work *



* Shallow Copy v/s Deep Copy
  + create new object But nested objects are still shared in memory.

Way :
1. spread operator {...obj}
2. Object.assign( )

Execution Context: Every function has its own execution context created
  + variable environment
  + scope chain
    + global scope
    + function Scope
    + Block Scope
      * only variable lookup allow
  + This keyword : dynamic in nature,

Context / value of this keyword depends on How Function is called.

* This in normal function: undefined
* Method Calling - This → caller object.
* This in Arrow → points its parent context
* Event listner : this point to event Attached on it

* To set the manully context of this
* (call(), apply(), Bind()) Method

Explicitly set the context of this keywords.

* Arguments keyword (Execution context)

[ ] Array of arguments

this hold all the arguments that we pass during Function calling.

* Deep Copy : create Completly New Object in memory Independent
  * JSON.parse(JSON.stringify(obj))
  * structuredClone(obj);

* memory management in javascript:

Life cycle:
1. Allocation (memory assign)
2. Read / write operation
3. Deallocation.

(garbage collections) → mark Algo, Sweep Algo.

* Reachability :
if value is reachable stay in memory, if not Removed